



COMPUTER SCIENCE 180A ALGORITHMS

PROGRAMMING ASSIGNMENT 1 – BRANDEIS MAP

Develop a comprehensive map utility for navigating Brandeis, modeled as a symmetric directed graph. This system will assist users in finding optimal paths and efficient campus tours based on real-world traversal considerations.

The implementation will include two core functionalities:

1. Shortest Path Navigation

Using **Dijkstra's algorithm**, compute the shortest path between any two points on campus. The algorithm must support two distinct optimization modes:

- **Minimize Distance:** Computes the path with the lowest total distance in feet.
- **Minimize Time:** Computes the path that can be traversed in the shortest amount of time, accounting for various traversal conditions.

a. Graph Structure

The campus is represented as a **symmetric directed graph**, meaning that for every directed edge ($u \rightarrow v$), there exists a corresponding edge ($v \rightarrow u$). This structure allows direction-specific attributes such as angle, direction and code, while maintaining connectivity in both directions.

Each edge in the graph is defined by the following attributes:

(EdgeID, startVertexID, destinationVertexID, length, angle, direction, code, name)

- **EdgeID:** A unique identifier for Edges
- **startVertexID / destinationVertexID:** The endpoints of the directed edge.
- **startVertexLabel/destinationVertexLabel:** The labels of the vertex endpoints
- **length:** The physical length of the edge in **feet**.
- **angle:** The map angle from the startVertex to destinationVertex where 0° represents the vertical up direction on the map, 90° east on the map and so on. (true angles can be computed by subtracting 25)
- **direction:** A cardinal direction for a human-readable indication of angle.
- **code:** A symbolic label that classifies the type of path and determines the adjustment factor used to compute walking speed (see table in section 1.b).
- **name:** A human-readable name for the path, such as "South Street" or "Loop Road".

The implementation of Dijkstra's algorithm should process any edge accordingly, using:

- **length values** when minimizing for distance.
- **Computed traversal time** based on the edge's code and adjusted speed when minimizing for time (see section 1.b).

b. Computation of Time

Based on 3.1 miles per hour as the average human walking speed on level ground (e.g., see Wikipedia), use this constant for the human walking speed on level ground:

$$\text{WalkSpeed} = (3.1 \text{ miles/hr}) * (5280 \text{ ft/mile}) / (60 \text{ mins/hr}) = 272 \text{ feet / minute}$$

For special kinds of edges, the speed at which the edge may be traversed must further be adjusted by these constants:

Code	Meaning	Factor
f	WalkFactorF	1
u	WalkFactorU	0.9
d	WalkFactorD	1.1
s	StepFactorU	0.5
t	StepFactorD	0.9
b	BridgeFactor	1
x	Undefined	1

The formula for computing time in seconds is defined as follows:

$$\text{time} = (\text{length in feet} / (\text{WalkSpeed} \times \text{factor})) \times 60$$

For example:

- A 770 ft uphill edge (code = 'u') would be traversed at $272 \times 0.9 = 244.8$ ft/min.
- The time to traverse it would then be: $(770 / 244.8) \times 60 \approx 188.7$ seconds.

The 'x' code currently behaves like flat ground (factor = 1.0) but is reserved for possible future use where more complex behaviors may be defined.

2. Efficient Campus Tour

To assist in planning efficient campus tours (e.g., for visitors or maintenance staff), the system will approximate a **Hamiltonian cycle** that visits every location (not including map corners).

This functionality is built in several steps:

a. Constructing a Minimum Spanning Tree (MST)

Implement both **Kruskal's and Prim's algorithm** to build a MST over the campus graph. Since both algorithms assume that the graph is undirected, they have to be modified to deal with the Brandeis map being modeled as a symmetric directed graph:

- **Graph Conversion for MST:** Treat each pair of directed edges ($u \rightarrow v$ and $v \rightarrow u$) as a single undirected edge between u and v .
- **Edge Weighting:** For the undirected edge (u, v) , define its weight as the **sum of the weights of the two directed components** (i.e., $\text{weight}(u \rightarrow v) + \text{weight}(v \rightarrow u)$), regardless of whether minimizing for time or distance.

b. Creating a cycle from MST

Once the MST is constructed, perform a **pre-order traversal** starting from the designated root node to create a cycle that has cost equal to the sum of all edges in the MST, (i.e., upon returning, preorder visits the edge in the opposite direction). For example, if the starting vertex was A for the following tree, a pre-order traversal would be A, B, C, B, A, D, A.

c. Tour Optimization

To improve the quality of the cycle:

- **Edge Replacement:** Look for pairs of successive edges in the tour that could be replaced by a single direct edge with a lower total cost (time or distance). For instance, if there was an edge $c \rightarrow a$ that was smaller than $c \rightarrow b + b \rightarrow a$, then the tour can be reduced by making that substitution. Ensure that every vertex is still visited at least once and that the tour finishes where it starts.

3. Provided Resources

You are provided with the following: LastNameFirstNameAssignment1.zip

This folder contains all the necessary starter code, data files, and map images to support the full implementation of the campus map utility. The structure of the contents is as follows:

File Name	Package	Purpose
Map.java	main	Main class for managing the overall campus map and graph construction.
Vertex.java	main	Class for a vertex (node) in the graph, representing a campus location. Additionally supports Disjoint-Set operations
Edge.java	main	Class representing a directed edge between two vertices.
Graph.java	main	Class representing a graph using an adjacency list representation
ConsoleIO.java	main	A class for handling console-based user interaction (e.g., prompting for input and outputting paths)
Minheap.java	datastructures	A custom min-heap implementation to support Dijkstra's, Prim's, and Kruskal's algorithm efficiently.
HashMap.java	datastructures	A custom hash map implementation, used to map Vertex/Edge/String to a Number.
ImagePathDisplay.java	visual	Provides GUI or visual functionality to display the path overlay on the campus map images.

Note: These Java files form the skeleton of the program, and it is expected that further functionality be implemented or extended to complete the assignment. **Do not delete or rename any of them.**

Map Images

File Name	Description
CampusMap.jpg	A full image of the college campus map.
CampusMapCropped.jpg	A cropped version of the map.

Text Files (Graph Data + Output)

File Name	Description
MapDataVertices.txt	Contains the list of all vertices in the graph (campus locations), including their coordinates and identifiers.
MapDataEdges.txt	Describes all the edges (paths) between vertices, including direction, length, and edge type (code).
Route.txt	Output file generated after constructing a path or a tour. Each line corresponds to an edge, formatted as four-pixel coordinates to draw a line segment on the campus map.
RouteCropped.txt	Same as Route.txt, but coordinates are relative to the cropped map image.
RouteEdges.txt	Textual representation of the edges on the computed path, showing the vertex endpoints for each edge.

Sample Inputs & Outputs

A folder is included with example input/output files for both functionalities of the project:

- **Shortest Path (Dijkstra) example**
- **Campus Tour (MST + Approx. TSP) example**

4. Methods to implement

main.Vertex.java

To implement Kruskal's algorithm the Vertex class must support disjoint-set operations. Every vertex therefore has two fields to help with implementation, Vertex.parent and Vertex.rank. The following four methods do not require any other additional fields or classes/datastructures outside of those two.

public void makeSet() - Initializes a set whose only member (and thus representative) is this vertex

public void union(Vertex y) - unites the two disjoint sets containing this vertex and the given vertex y such that both vertices will belong to the same set (and thus share the same representative), uses union by rank heuristic

`public void link(Vertex y)` - links two trees rooted at this vertex and the given vertex `y`, where both trees are roots of different sets, using union by rank heuristic

`public Vertex findSet()` - returns the Vertex representative of the set containing this vertex and uses the path compression heuristic

main.Edge.java

`public double priority(boolean optimizingTime)` - returns the length of the edge in feet or the time it takes to traverse it in seconds based on the value of optimizing Time

main.Map.java

`private Graph minimumSpanningTreeKruskals(boolean optimizingTime)` - returns a graph that represents the undirected minimum spanning tree as discussed in section 2. Recall that Kruskal's algorithm works as follows:

```
MST-Kruskal(Graph, weight)
1. A = empty set
2. for each vertex v in Graph.V
3.     Make-Set(v)
4. Create a single list of the edges in Graph.E
5. Sort the list of edges into monotonically increasing order by weight
6. for each edge (u, v) taken from the sorted list in order
7.     if Find-Set(u) != Find-Set(v)
8.         A = A union {(u, v)}
9.         Union(u, v)
10. return A
```

`private Graph minimumSpanningTreePrims(boolean optimizingTime)` - returns a graph that represents the undirected minimum spanning tree as discussed in section 2. Recall that Prim's algorithm works as follows:

```
MST-Prim(Graph, weight, start)
1. for each vertex u in Graph.V
2.     u.key = infinity
3.     u.prev = NIL
4. Start.key = 0
5. Q = empty set
6. for each vertex u in Graph.V
7.     insert(Q, u)
8. while Q not empty set
9.     u = extract_min(Q)
10.    for each edge (u, v) in Graph.E
11.        if v in Q and weight(u, v) < v.key
12.            v.prev = u
13.            v.key = weight(u, v)
14.            Decrease-Key(Q, v, weight(u, v))
```

`public Edge[] preorderTraversal(Graph tree, int startVertexID)` - returns an array of edges that represents the preorder traversal of the tree as specified in part 2.

`public Edge[] replaceSuccessiveEdges(Edge[] tour, boolean optimizingTime)` - returns an array of edges that implements shortcutting by replacing two successive edges in the tour with one edge if the new edge is faster/shorter than the two edges combined and if the vertex common to both edges is visited at least once.

`public Edge[] dijkstra(int start, int destination, boolean optimizingTime)` - returns an array of edges that represents the fastest/shortest path from start to destination. Recall that Dijkstra's algorithm works as follows:

```
Dijkstra(Graph, weight, start)
1. for each vertex v in Graph.V
2.   v.d = infinity
3.   v.pred = nil
4. Start.d = 0
5. S = empty set
6. Q = empty set
7. for each vertex u in Graph.V
8.   insert(Q, u)
9. while Q not empty set
10.  u = extract_min(Q)
11.  S = S union {u}
12.  for each edge (u, v) in Graph.E
13.    if v.d > u.d + weight(u, v)
14.      v.d = u.d + weight(u, v)
15.      v.prev = u
16.      Decrease-Key(Q, v, v.d)
```